
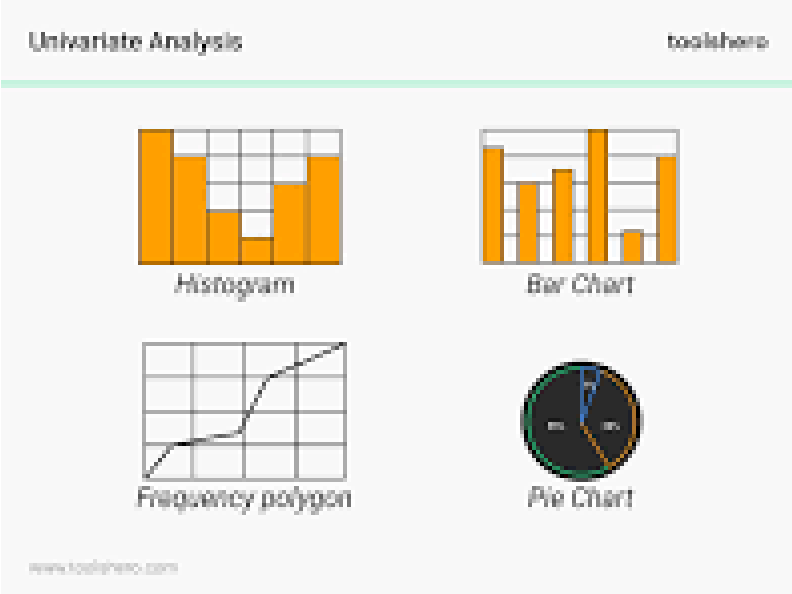

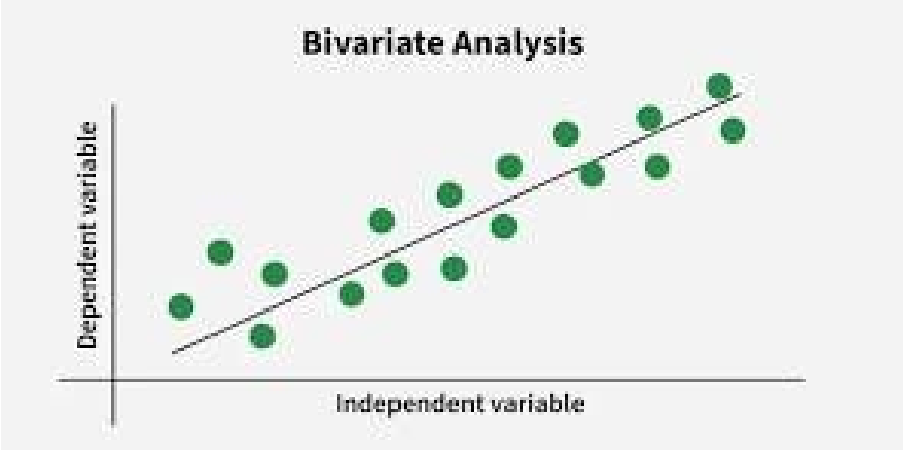

Univariate and bivariate analysis break down a dataset by **examining single features and connections between pairs of variables**.
Univariate Analysis (One Variable)

Univariate analysis looks at one variable at a time to describe its individual properties. It acts as the foundation of descriptive statistics.

- **Key Focus:** Finds the center, spread, and shape of a single data column.
- **Core Metrics:**
  - **Central Tendency:** Mean, median, and mode show the typical or middle value.
  - **Dispersion:** Range, variance, and standard deviation show how spread out the data points are.
- **Common Visuals:** Histograms, box plots, and bar charts.
- **Role in Understanding Data:** It highlights basic data profiles, data entry errors, and outliers before deeper testing begins.

Bivariate Analysis (Two Variables)

Bivariate analysis looks at two variables at the same time to see how they relate to each other. It moves past simple description into finding associations.

- **Key Focus:** Determines if a connection exists, how strong it is, and which direction it goes.
- **Core Metrics & Methods:**
  - **Correlation Coefficients:** Measures the linear link between two numbers on a scale from -1 to 1.
  - **Regression Analysis:** Creates a mathematical model to predict one variable using the other.
  - **Crosstabulation & Chi-Square:** Tests associations between category-based traits.
- **Common Visuals:** Scatter plots and line graphs.
- **Role in Understanding Data:** It uncovers dependencies, tests research hypotheses, and reveals how one factor influences another.

Training error is the mistake rate a model makes on the data it learned from, while test error is the mistake rate it makes on completely new, unseen data. Evaluating a model on unseen data is essential because **a low training error only proves a model can memorize its training set, whereas test error reveals its true ability to generalize to the real world**.

Training Error vs. Test Error

The core differences between the two metrics include:

Metric	Data Used	Main Purpose
Training Error	**Training Set** (Data used to fit model parameters)	Measures how well the model **learns basic patterns**.
Test Error	**Test Set** (Separate, unseen data)	Measures how well the model **generalizes to new inputs**.

Why Performance Must Be Evaluated on Unseen Data

- **Detects Overfitting:** Highly flexible models can easily minimize training error to zero by memorizing specific data points, random fluctuations, and background noise. Unseen data exposes whether a model actually understands underlying concepts or has just memorized the answers.
- **Provides an Unbiased Metric:** Training error is inherently optimistic. Evaluating performance on a distinct test set provides an honest, unbiased estimation of how the model will perform in production.
- **Balances Bias and Variance:** Tracking test error helps data scientists locate the optimal level of model complexity. It highlights the "sweet spot" where a model is complex enough to capture the real trends without becoming overly sensitive to training noise.

K-fold cross-validation is a data resampling technique used to evaluate the true generalization ability of a machine learning model by splitting a single dataset into k equal-sized subsets (folds). The model is trained and tested k times; in each iteration, a unique fold acts as the testing/validation set while the remaining $k-1$ folds form the training set. The final performance estimate is the average score across all k iterations.

How the Process Works Step-by-Step

According to standard evaluation guidelines detailed by Machine Learning Mastery, the process follows a strict loop:

1. **Shuffle:** Randomly reorder the dataset to ensure data points are well distributed.
2. **Split:** Partition the dataset into k equally sized subsets (common values are $k=5$ or $k=10$).
3. **Iterate:** For each individual fold i from 1 to k:
    - Hold out fold i to serve as the temporary validation/test dataset.
    - Train the model using all remaining $k-1$ folds.
    - Test the trained model on fold i and record its performance score.
4. **Aggregate:** Average the performance metrics from all k iterations to calculate the final cross-validation score.

Why K-Fold Provides a More Reliable Performance Estimate

Relying on a single train-test split (the holdout method) often yields an unstable performance estimate. K-fold cross-validation mitigates these limitations in several ways:

- **Eliminates "Selection Bias" (Split Luck):** In a traditional single split, a model might score artificially high if the test set accidentally contains mostly "easy" examples, or artificially low if it contains "difficult" exceptions. K-fold eliminates this variability by ensuring that **every single data point** is used for testing exactly once, and for training $k-1$ times.
- **Maximizes Data Efficiency:** With limited datasets, a static train-test split forces a trade-off: using more data for training leaves too little data for a stable test score. K-fold allows the model to utilize 100% of the data across the entire execution cycle for both training and validation roles.
- **Provides a Measure of Variance:** Beyond giving a single average score, k-fold allows practitioners to calculate the standard deviation across folds. A low standard deviation confirms model stability, while a high standard deviation signals that the model is highly sensitive to the specific data it is trained on (high variance/overfitting).
- **Prevents Overfitting in Model Tuning:** When used for hyperparameter tuning or model selection, relying on multiple rotating validation sets ensures that selected parameters generalize broadly rather than anchoring to a single fixed split.

---

1. Structured Data:
Data organized in a fixed format such as rows and columns.
Example: Student database or Excel table.

2. Semi-structured Data:
Data that does not follow a strict tabular structure but contains tags or keys.
Example: JSON and XML files.

3. Unstructured Data:
Data without a predefined structure.
Example: Images, videos, audio files, and text documents.

What is iris data set? Explain.
The Iris dataset is **a famous, classic multi-class dataset introduced by biologist and statistician Ronald Fisher in 1936**, often considered the "Hello World" of machine learning and data science for practicing classification algorithms.

It contains 150 total rows with 50 samples from each of three iris flower species:

- *Iris setosa*
- *Iris versicolor*
- *Iris virginica* [6]

Columns (Variables)

The dataset typically consists of five main columns (four feature measurements and one target label):

- **Sepal Length**: The length of the iris sepal measured in centimeters.
- **Sepal Width**: The width of the iris sepal measured in centimeters.
- **Petal Length**: The length of the iris petal measured in centimeters.
- **Petal Width**: The width of the iris petal measured in centimeters.
- **Species (or Class)**: The identifier or target category for the flower (*Iris-setosa*, *Iris-versicolor*, or *Iris-virginica*)*(Note: Some versions available on platforms like Kaggle also include an initial column).*

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Plotting

```
# Syntax
hist(x,
    breaks = "Sturges",   # Number of bins or method
    col = "color_name",   # Fill color
    border = "color_name",# Border color
    main = "Title",       # Main title
```

```
    xlab = "X-axis label",# X-axis label
    ylab = "Y-axis label" # Y-axis label
)

# Sample data
data <- c(5, 7, 8, 5, 6, 9, 12, 15, 18, 20, 22, 25)

# Plot boxplot
boxplot(data, col = "lightgreen", main = "Box Plot Example", ylab = "Values")
```